COMP5331 Knowledge Discovery in Databases (Fall Semester 2026)
Homework 1 Solution

Q1

(a)
Yes. It can be adapted.

First of all, we obtain the 2-sequence by scanning the database.
(NOTE: The 2-sequence contains two kinds of sequences – (1) the sequence contains only one *timestamp entry* (e.g. <{D, E}>) and (2) the sequence contains two or more timestamp entries (e.g. <{D}, {E}>).

The Apriori-like algorithm is described as follows.
1. k=2
2. Find all frequent 2-sequences and store them in L_k
3. repeat
4. k=k+1
5. Generate candidate k-sequences from L_{k-1} (which will be described later) and store them in C_k
6. Scan the database and count the support of each candidate in C_k
7. Find the k-sequence in C_k with support $\geqq$ minsupport and store them in L_k
8. until L_k = empty set
9. return L_i for i=2,…k

e.g.
We obtain the following 2-sequences.
{<{A}, {D}>, <{A}, {E}>, <{A}, (G}>, <{D, E}> }

Next, we generate the candidate 3-sequences by the join-and-prune process.

The *join* step of the generation process is described as follows.
A sequence $s^{(1)}$ is joined with another sequence $s^{(2)}$ only if the subsequence obtained by dropping the first item in $s^{(1)}$ is identical to the subsequence obtained by dropping the last item in $s^{(2)}$. The resulting candidate is the sequence $s^{(1)}$, concatenated with the last item from $s^{(2)}$. The last item from $s^{(2)}$ can either be joined into the same timestamp element as the last item in $s^{(1)}$ or different timestamp elements depending on the following conditions.
1. If the last two items in $s^{(2)}$ belong the same timestamp element, then the last item in $s^{(2)}$ is part of the last timestamp element in $s^{(1)}$ in the joined sequence. (e.g. Suppose we have frequent sequences <{A},{B},{C}> and <{B},{C, D}> in L_{k-1}. Candidate <{A},{B},{C, D}> is obtained by joining <{A},{B},{C}> and <{B), {C, D}>).
2. If the last two items in $s^{(2)}$ belong to different timestamp elements, then the last item in $s^{(2)}$ becomes a separate timestamp element appended to the end of $s^{(1)}$ in the joined sequence. (e.g. Suppose we have frequent sequences <{A},{B},{C}> and <{B},{C},{D}> in L_{k-1}. Candidate <{A},{B},{C},{D}> is obtained by joining <{A},{B},{C}> and <{B},{C},{D}>).

e.g. In the running example, we obtain one candidate 3-sequence <{A}, {D,E}> (by joining <{A}, {D}> and <{D, E}>) after the join step.

The *prune* step of the generation process is described as follows.
A candidate k-sequence is pruned if at least one of its (k-1)-sequence is infrequent.

For example, <{A}, {D,E}> is a candidate 3-sequence. We need to check whether <{A}, {E}> is a frequent 2-sequence (NOTE: We do not need to check whether <{A}, {D}> and <{D, E}> are frequent 2-sequence because <{A}, {D,E}> was generated from these two frequent sequences). Since <{A}, {E}> is frequent, <{A}, {D,E}> is also considered as a candidate 3-sequence after the prune step.

Then, we do the *counting* step to count the support of each candidate in the set.
As the support of <{A}, {D,E}> is 2, then it is one of the final results.

We repeat the process until L_k is an empty set.

In our running example, all sequences with support at least 2 are {<{A}, {D,E}> }

(b)
No. This is because if a k-sequence has its important ratio at least 2, it is not necessarily true that all its sub-sequences have their important ratios at least 2.

Consider an example with the following 4 transactions.

Customer W, Rich = Yes, time 1, items A, B, C
Customer X, Rich = Yes, time 2, items A, B, C
Customer Y, Rich = No, time 3, items A, B, C
Customer Z, Rich = No, time 4, items A, B

The above transactions can be transformed into four sequences as follows.
W, Yes: <{A, B, C}>
X, Yes: <{A, B, C}>
Y, No: <{A, B, C}>
Z, No: <{A, B}>

The supports of <{A, B, C}> with respect to values “Yes” and “No” are 2 and 1, respectively.
Thus, the important ratio of <{A, B, C}> is 2/1 = 2.

Consider a sub-sequence of <{A,B,C}>, i.e., <{A, B}>.
The supports of <{A, B}> with respect to values “Yes” and “No” are both 2.
Thus, the important ratio of <{A, B}> is 2/2 = 1 < 2.

Note that <{A, B}> is a subsequence of <{A, B, C}> but the important ratio of <{A, B}> is smaller than the important ratio of <{A, B, C}>. Thus, the apriori property is not satisfied. Thus, we cannot adapt the Apriori algorithm.

Q2

(a)
(i)
Adaptions:

1. The major adaption is that we define a new concept of "segment" which corresponds to a sequence of several consecutive items *with the same support* along a path in a conditioned FP-tree. This helps to prune a lot of pattern candidates which cannot be "closed" itemsets.
2. Another adaption is that we maintain a set containing the set generated closed itemsets, based on which, we can obtain a lower bound of the k-th greatest support of the closed itemsets. Also note that this lower bound is used to prune some closed itemsets and is *dynamically* maintained.

Pseudo-code:
First, we will give a procedure FP-growth and then give the main algorithm.

Algorithm FP-growth (Tree, α)
$\mathbf{A} \leftarrow \emptyset$; //The set containing the final results.
$\mathbf{B} \leftarrow \emptyset$; //The set containing some candidates of the final results.
thr $\leftarrow$ 1; //The k-th greatest support of closed itemsets, which is dynamically maintained.
if Tree contains a single path P
Partition P into different segments such that all nodes in each segment have the same node support;
for each segment S
Choose the node N in S at the deepest;
$\beta \leftarrow$ The set of nodes from N to the root;
Generate the pattern $\beta \cup \alpha$ with support equal to min.support of nodes in β;
If ∃ an itemset I in **A** such that $\beta \cup \alpha$ is a subset of I and I.count = ($\beta \cup \alpha$).count
thr $\leftarrow$ max{thr, the k-th greatest support of the itemsets in **A};**
If ($\beta \cup \alpha$).count $\geq$ *thr*
Insert $\beta \cup \alpha$ into **A**;
else
for each a_i in the header of Tree
Generate pattern $\beta = a_i \cup \alpha$ with support=a_i.support;
Insert β into **B**;
Construct β's conditional pattern base and β's conditional FP-tree $Tree_{I2}$;
if $Tree_\beta \neq \emptyset$
FP-growth ($Tree_{I2}$,β);

Algorithm main

1. Build an FP-tree Tree
2. Call FP-growth (Tree, ∅)
3. Perform the following
   Process itemsets in B in descending order of the size of the itemsets;
   for each itemset I_B in **B**
   if ∃ an itemset I_A in **A** such that I_A is superset of I_B and I_A.count = I_B.count
   thr $\leftarrow$ max{thr, the k-th greatest support of the itemsets in **A};**
   If ($\beta \cup \alpha$).count $\geq$ *thr*

Insert I_B into **A**;
thr ← the k-th greatest support of the itemsets in **A;**
Remove from **A** all those itemsets with suppor t smaller than *thr*.
Output **A**.

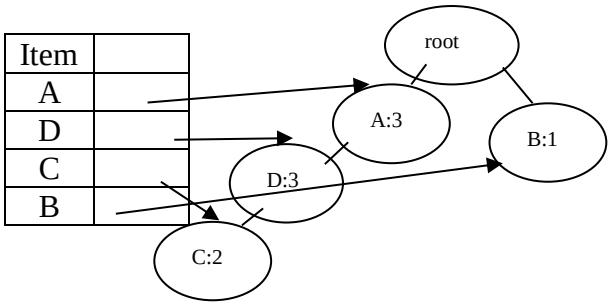
(ii)
Illustration: (ordered)

Item	Freq.
A	3
B	1
C	2
D	3

Item	Freq.
A	3
C	2
D	3
B	1

Item	Freq.
A	3
D	3
C	2
B	1

FP-tree



Ans: {ACD: 2, AD: 3}.
Illustration:
We process the headers B, C, D and A in order.
When processing item B, we first generate pattern β= (B:1) and put it into **B**.
Along the conditional FP-tree of β, we have no segments and thus we generate nothing in **A**.
At this moment, **A** = {} and **B** = { B:1}.

When processing item C, we first generate pattern β= (C:2) and put it into **B**.
Along the conditional FP-tree of β, which is a single path D-A-root, we have only one segment, i.e. , D-A, with the corresponding support equal to 3.
Thus, we generate pattern (CDA:2) and put it into **A**.
At this moment, **A** = {CDA:2} and **B** = {C:2, B:1}.

When processing item D, we first generate pattern β= (D:3) and put it into **B**.
Along the conditional FP-tree of β, which is a single path A-root, we have only one segment, i.e. , A, with the corresponding support equal to 3.
Thus, we generate pattern (DA:3) and put it into **A**.
At this moment, **A** = {DA:3, CDA:2} and **B** = {D:3, C:2, B:1}.

When processing item A, we first generate patternβ= (A:3) and put it into **B**.
Along the conditional FP-tree of β, which is a single path root, we have no segments.
At this moment, **A** = {DA:3, CDA:2} and **B** = {A: 3, D:3, C:2, B:1}.

Now, we perform a post-processing on **B**.
(A: 3) is not included in **A** since (DA:3) is in **A**, which is a superset of (A: 3), has the same support of (A:3).
(D:3) is not included in **A** for the same reason.
(C:2) is not included in **A** for the same reason.
(B:1) is included in **A** since it is closed.

As a result, **A** becomes {DA:3, CDA:2, B:1}.

Finally, we perform to remove some closed itemsets from **A** each of which has its support less than the k-th greatest support of the closed itemsets (*thr* = 2).
This step excludes (B: 1) from **A** and thus the final result is {DA:3, CDA:2}.

(iii)
Ans: {ACD: 2, AD: 3, B:1}.
Illustration:
The illustration for the case of k=3 is similar to that for the case of k=2 except for the last step.

(b) Advantages:

1. The number of possible frequent closed itemsets is much smaller than that of frequent itemsets, so the storage is smaller.
2. We do not need to generate all possible frequent itemsets, so the execution time is shorter.
3. Parameter K is more user-friendly compared with the support threshold parameter.

Disadvantages:

1. We need to expand the closed frequent itemset in order to check whether an itemset is frequent or not.
2. It is not easy for users to understand the closed frequent itemsets directly.

Q3

(a)

(i) Cluster1: mean=(9.2857, 9.5714) points: $x_1, x_2, x_3, x_4, x_5, x_6, x_7$

Cluster2: mean=(10.0000, 30.0000) points: x_8

(ii) Cluster1: mean=(4.2500, 8.0000) points: x_2, x_4, x_6, x_7

Cluster2: mean=(14.5000, 16.2500) points: x_1, x_3, x_5, x_8

(iii) Cluster1: mean=(14.5, 9) points: x_1, x_3, x_5, x_7

Cluster2: mean=(10, 30) points: x_8

Cluster3: mean=(2.33, 10.33) points: x_2, x_4, x_6

(iv) Cluster1: mean=(10, 1) points: x_7

Cluster2: mean=(10, 30) points: x_8

Cluster3: mean=(2.3333, 10.3333) points: x_2, x_4, x_6

Cluster4: mean=(16, 11.6667) points: x_1, x_3, x_5

(b) Advantages:

1. It is easy to implement.
2. It is easy to understand.

Disadvantages:

1. It is difficult to determine the value of k because we do not know how many clusters.
2. K-mean is sensitive to the initial guess of the means.

For each disadvantage, we can overcome with the following suggestion.

i. We can try different values of k and adopt the most suitable value of k.

ii. We can try different possible sets of initial guesses.

Q4

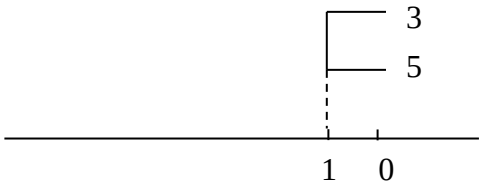
(a)

	1	2	3	4	5	6	7	8
1	0							
2	11	0						
3	5	13	0					
4	12	2	14	0				
5	7	17	1	18	0			
6	13	4	15	5	20	0		
7	9	15	12	16	15	19	0	
8	11	20	12	21	7	22	30	0

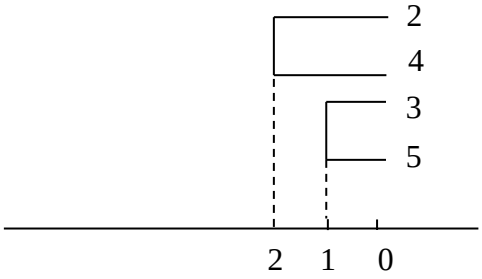
Distance

	1	2	(35)	4	6	7	8
1	0						
2	11	0					
(35)	6	15	0				
4	12	2	16	0			
6	13	4	17.5	5	0		
7	9	15	13.5	16	19	0	
8	11	20	14.5	21	22	30	0

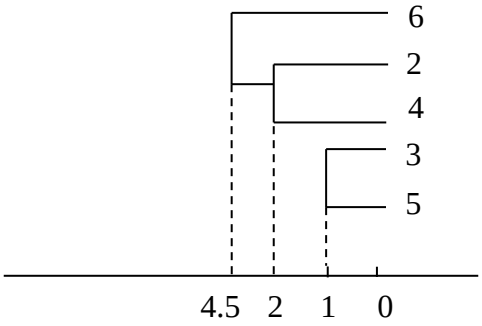
Dendrogram



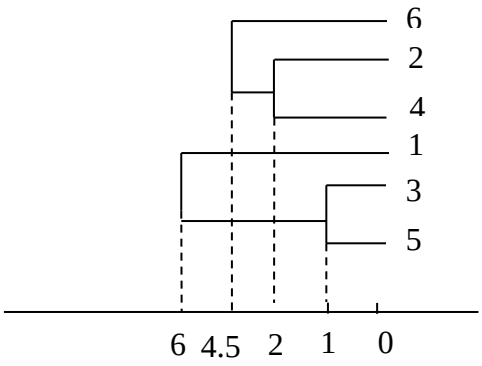
	1	(24)	(35)	6	7	8
1	0					
(24)	11.5	0				
(35)	6	15.5	0			
6	13	4.5	17.5	0		
7	9	15.5	13.5	19	0	
8	11	20.5	14.5	22	30	0



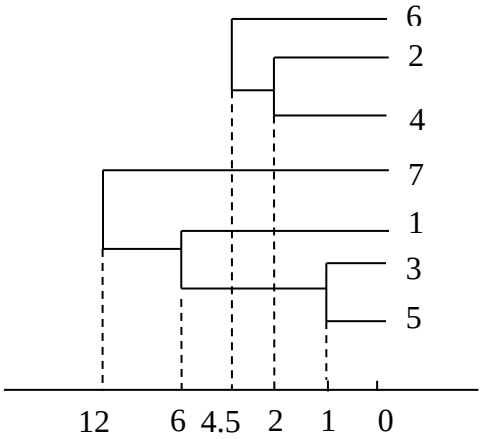
	1	(246)	(35)	7	8
1	0				
(246)	12	0			
(35)	6	16.17	0		
7	9	16.67	13.5	0	
8	11	21	14.5	30	0



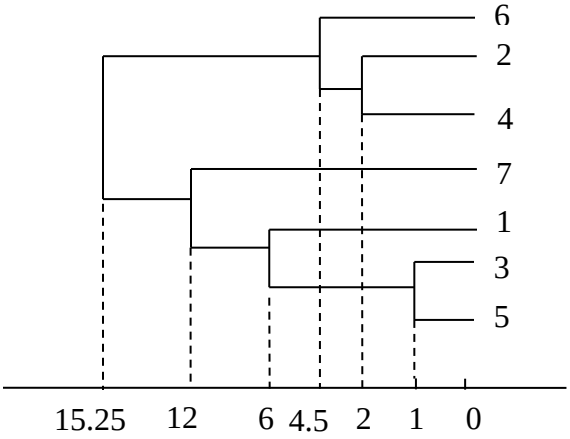
	(135)	(246)	7	8
(135)	0			
(246)	14.78	0		
7	12	16.67	0	
8	13.33	21	30	0

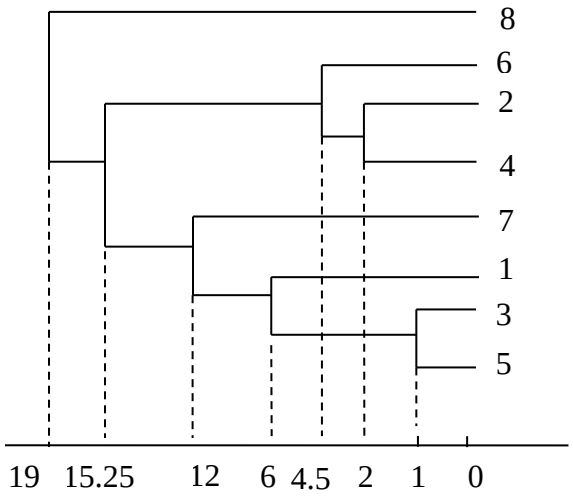


	(1357)	(246)	8
(1357)	0		
(246)	15.25	0	
8	17.5	21	0



	(12345467)	8
(1234567)	0	
8	19	0





(b)
Cluster 1 = {2, 4, 6}
Cluster 2 = {1, 3, 5}
Cluster 3 = {7}
Cluster 4 = {8}

(c)
(i)
8

(ii)
3

(d)
Yes. It can improve the computation efficiency. Note that every time, we merge two clusters at an iteration. We need to re-compute some pairwise distances. We can make use of the triangle inequality to reduce some re-computations.

If the triangle inequality is satisfied, we can improve the performance of the agglomerative approach. Specifically, for each cluster, we store the distance between two distant data points in this cluster. Besides, for each cluster, we also store the shortest distance from this cluster to the other cluster.
Consider three clusters, C1, C2 and C3.
C1 has the distance between two distant data points as $d_{1,max}$ and the shortest distance to the other cluster as $d_{1,min}$.
C2 has the distance between two distant data points as $d_{2,max}$ and the shortest distance to the other cluster as $d_{2,min}$.
C3 has the distance between two distant data points as $d_{3,max}$ and the shortest distance to the other cluster as $d_{3,min}$.

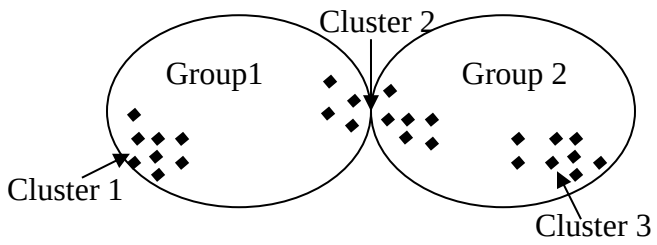
If we know that $d_{3,min}$ is larger than $d_{2,min}+d_{1,max}$ (which corresponds to the greatest possible maximum distance between C1 and C2), then we do not need to compute the distance between C1 and C3. This is because we know that the smallest possible distance between C1 and C3 (i.e., $d_{3,min}$) is larger than the greatest possible maximum distance between C1 and C2. We know that the distance between C1 and C3 must not be used for the cluster pair selection at iteration.

Q5

(a) No. This is because this algorithm terminates when k is equal to the total number of data points. In this case, e_k is equal to 0. Even if k is larger, e_k is also equal to 0 and thus e_k converges. The value k is not what we desire because we want to group "similar" points.
We define S(p) for a data point p as S(p)=(b-a)/max(a, b), where a is the mean intra-cluster distance (from p to other points in the same cluster) and b is the mean nearest-cluster distance (from p to all the points in the "nearest cluster" that p is not part of), which can also be thought as a type of inter-cluster distance. S(p)=0 if p is the only one point in the its cluster. The "nearest cluster" of p is set to the one with the smallest mean distance from p to all the points in it. It is also called Silhouette Coefficient. Let d_k be the average Silhouette Coefficient over all data points.
Alg:
a. First, compute d_k.
b. We change Step 4 from e_k to d_k .
c. We change Step 5 to the following:
We repeat Step 3 to Step 4 for different possible values of k and obtain the corresponding values of d_k.
d. We find the k s.t. d_k is maximized.

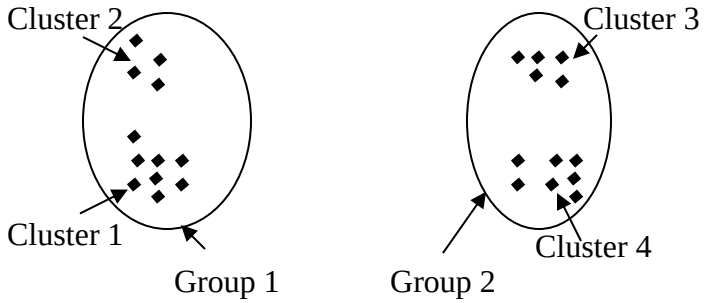
The reasons are:
1. If k is larger than the number of clusters, in k-means, one single cluster will be represented by more than one means. In other words, the real cluster is split into a number of groups in k-means. The inter-cluster distance b will be small since some close points that should be put in the same group are put into different groups. It is expected that these groups are very close and thus d_k is small.
2. If k is smaller than the number of clusters, in k-means, two or more clusters will be represented by one mean/group.
There are two cases:

- Case 1:

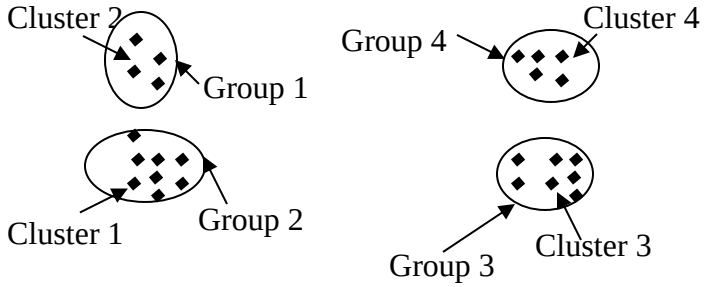


One cluster is split into two or more groups.
In the example, when k=2, cluster 2 is split into group 1 and group 2. It is easy to see that d_k is small since the inter-cluster distance b is small and the intra-cluster distance a is large.

- Case 2:



In this example, when k=2, cluster 1 and cluster 2 completely belong to group 1 while cluster 3 and cluster 4 completely belong to group 2.
In this case, we know that the separation between group 1 and group 2 is large and thus d_k is large. However, its intra-cluster distance a is also large.
When k=4, we have



Obviously, d_k in this case is larger because its intra-cluster a is smaller.
In conclusion, if we find k s.t. d_k is maximized, we can find a good value for the number of clusters.

(b) The distance metric between a point p and the cluster center c can be modified as follows.
Let X_p be a "special" cluster containing only p.
Let X_c be the cluster representing the cluster center c which contains some assigned points.
The first assigned point of X_c is the point closest to c for each c.
The distance metric between a point p and the cluster enter c is equal to the distance between X_p and X_c according to the single linkage distance in an iterative manner.
While there are still remaining points, find a remaining point p and a cluster X such that the distance between p and X is the shortest and assign p to cluster X.